



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验八：VGA 图形输出与键盘输入

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解 UEFI Graphics Output Protocol (GOP) 及线性 framebuffer 的工作方式。
2. 在 Bootloader 中获取显示模式、分辨率、stride、像素格式和显存地址，并将其传递给内核。
3. 使用 `embedded-graphics` 抽象实现基本图元、字符绘制和 Shell 图形终端。
4. 将 framebuffer 输出接入内核日志及用户程序标准输出，同时保留串口调试能力。
5. 实现 PS/2 键盘中断，将 GUI 键盘与串口输入汇入同一 Shell 输入队列。
6. 完成上下分屏和动态时钟图形加分项。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU 8.2.2
图形接口	UEFI GOP，线性 framebuffer
图形库	<code>embedded-graphics 0.8.2</code>
键盘库	<code>pc-keyboard 0.8.0</code>
实测模式	1280×800，BGR，stride 1280
报告工具	Typst

三. 实验内容概述

本实验在原有串口控制台的基础上加入图形显示和键盘输入。完整数据通路如下：

UEFI GOP

```
-> Bootloader 获取 framebuffer 元数据
-> BootInfo::frame_buffer
-> FrameBufferConsole
|-- DrawTarget<Rgb888>
|-- embedded-graphics 字符和图元
|-- ANSI 清屏及日志颜色
`-- 上半屏动态时钟 / 下半屏 Shell

串口 RX IRQ4 ----\
      +-> INPUT_BUF<ArrayQueue<u8>> -> sys_read -> Shell
PS/2 IRQ1 -----/

内核日志 / sys_write
-> print_internal
|-- UART16550
`-- FrameBufferConsole
```

这种设计没有修改用户程序的标准 I/O 接口。Shell 仍然通过 `stdin`、`stdout` 和系统调用工作，显示设备与输入设备的合并完全由内核完成。

四. 实验原理

1. UEFI Graphics Output Protocol

UEFI GOP 提供当前显示模式和 framebuffer。Bootloader 使用：

```
let handle = boot::get_handle_for_protocol::<GraphicsOutput>()?;
let mut gop = boot::open_protocol_exclusive::<GraphicsOutput>(handle)?;
let mode = gop.current_mode_info();
let mut frame_buffer = gop.frame_buffer();
```

本实验需要保存以下数据：

字段	作用
<code>address</code>	framebuffer 物理基地址
<code>size</code>	显存可访问字节数，用于边界检查
<code>width</code> / <code>height</code>	屏幕可见分辨率
<code>stride</code>	每条扫描线实际像素数，可能大于 width
<code>pixel_format</code>	RGB、BGR 或不支持的格式

像素地址不能简单使用 `y * width + x`，必须使用：

$$\text{offset} = (y \times \text{stride} + x) \times \text{bytes_per_pixel}$$

实测 QEMU GOP 模式为 1280×800、BGR、每像素 4 字节，framebuffer 物理地址为 `0x80000000`，大小为 `0x3e8000`。

2. framebuffer 地址映射

Bootloader 已经将物理地址空间线性映射到高半区。GOP 提供的是显存物理地址，因此内核访问地址为：

$$\text{virtual_address} = \text{framebuffer_address} + \text{physical_memory_offset}$$

显示驱动直接使用同一个 `BootInfo` 中的物理偏移，避免依赖全局地址模块的初始化顺序。所有显存读写使用 `read_volatile` 和 `write_volatile`，防止编译器消除具有设备副作用的访问。

3. embedded-graphics DrawTarget

`FrameBufferConsole` 实现：

```
impl OriginDimensions for FrameBufferConsole { ... }
impl DrawTarget for FrameBufferConsole {
    type Color = Rgb888;
    type Error = Infallible;
    ...
}
```

`draw_iter` 将 `Rgb888` 转换成 GOP 的 RGB/BGR 字节顺序，然后写入显存。实现 `DrawTarget` 后，字符、矩形、圆和直线都可以复用 `embedded-graphics` 的统一绘制接口。

字符渲染使用 `FONT_8X13` 单色字体。中文等字体中不存在的字符显示为 `?`，避免错误解析 UTF-8 字节。当前终端每个字符占 8×13 像素，行距为 15 像素。

4. 图形终端

屏幕被划分为两个区域：

- 上半屏：深蓝色图形仪表区，绘制标题、圆形表盘和动态秒针；
- 下半屏：深色 Shell 终端区，支持字符、换行、回车、退格、滚屏和清屏。

终端实现了实验所需的常用 ANSI 控制序列：

- `ESC[2J`：清除下半屏终端；
- `ESC[H`：光标回到终端左上角；

- `ESC[31m` 至 `ESC[36m` 、 `ESC[90m` ：日志颜色；
- `ESC[0m` ：恢复默认颜色。

当光标到达终端底部时，驱动按一行高度把 framebuffer 内容向上复制，再清除最后一行。Shell 的 `clear` 命令只清理下半屏，上方图形区继续保留。

5. 输出复用与并发

内核 `print_internal` 在关闭中断的临界区内依次写入 UART 和 framebuffer，因此内核日志、`sys_write`、Shell 及所有用户程序都会同时输出到两个设备。

framebuffer 由 `spin::Mutex` 保护。时钟中断更新图形时使用 `try_lock`：若普通输出正在持锁，就跳过当前动画帧，避免在中断上下文中等待锁形成死锁。串口仍然保留，图形驱动失效时也可以继续调试系统。

6. PS/2 键盘中断

PS/2 键盘使用 IRQ1，对应 IDT 向量 `0x21`。中断处理过程为：

1. 从 I/O 端口 `0x60` 读取扫描码；
2. 使用 `pc-keyboard` 的 `ScancodeSet1` 和 `Us104Key` 布局解析按键；
3. 将可表示为 ASCII 的 `DecodedKey::Unicode` 写入输入队列；
4. 向 Local APIC 发送 EOI。

键盘和串口接收中断都调用已有的 `drivers::input::push_key`。因此 `Resource::Console(Stdin)` 无须区分输入来源，用户态 `stdin().read_line()` 也不需要修改。

五. 实验过程与实现

1. 扩展 BootInfo

在 boot crate 中定义与 UEFI 生命周期无关的纯数据结构：

```
pub struct FrameBufferInfo {
    pub address: u64,
    pub size: usize,
    pub width: usize,
    pub height: usize,
    pub stride: usize,
    pub pixel_format: FrameBufferPixelFormat,
}
```

Bootloader 在退出 Boot Services 前获取 GOP 信息，并把 `Option<FrameBufferInfo>` 写入 `BootInfo`。若环境不提供 GOP，内核仍可依靠串口启动。

2. 实现图形驱动

新增 `drivers/framebuffer.rs`，实现以下能力：

1. RGB/BGR 像素转换和显存边界检查；
2. `embedded-graphics` 的 `DrawTarget<Rgb888>`；
3. `FONT_8X13` 字符绘制；
4. 换行、回车、退格和自动滚屏；
5. ANSI 清屏、光标复位和日志颜色；
6. 上下分屏；
7. 圆、直线、矩形等基本图形绘制；
8. 时钟中断驱动的动态秒针。

显示初始化在日志系统之前完成。日志初始化后会输出 `framebuffer` 模式确认信息，实测为：

```
Framebuffer Initialized: 1280x800, stride 1280, Bgr
```

3. 接入日志和标准输出

原有 `print_internal` 只调用 UART 的 `write_fmt`。修改后同时调用：

```
if let Some(mut serial) = get_serial() {
    serial.write_fmt(args).unwrap();
}
framebuffer::write_fmt(args);
```

用户态 `stdout` 经过 `sys_write` 到达内核 `Resource::Console`，最终同样进入 `print!`，所以无需修改 Shell 的普通输出逻辑。Shell 原有的 ANSI 清屏命令也可以直接由图形终端解释。

4. 实现键盘 IRQ

新增 `interrupt/keyboard.rs`，注册 IDT 向量 `0x21`，并在 IOAPIC 中启用 IRQ1：

```
keyboard::register_idt(&mut idt);
enable_irq(1, 0);
```

扫描码解码后写入和串口相同的 `ArrayQueue<u8>`。回车、退格、Shift 组合和常见符号由 `pc-keyboard` 状态机处理，Shell 继续使用原来的逐字节读取及回显逻辑。

5. 图形加分项

屏幕上半部分使用 `embedded-graphics` 绘制标题、表盘和秒针；下半部分专用于 Shell。时钟中断每 64 tick 尝试刷新一次秒针，降低在中断上下文中重绘大块区域的成本。

本实现没有额外创建用户态后台进程，而是把动态仪表作为低频内核显示任务。它同样可以在 Shell 运行时持续更新，并通过 `try_lock` 与终端输出安全共存。

6. QEMU 启动方式

默认 `make run` 改为保留 GUI，并把串口绑定到宿主终端：

```
QEMU_OUTPUT := -serial stdio
```

同时增加 `make run-headless`，需要纯串口模式时仍可使用 `-nographic`。

六. 测试与结果

1. 编译检查

以下命令均通过：

```
cargo check -p yyos_boot --lib --offline
cargo check -p yyos_kernel --lib --offline
make target/x86_64-unknown-uefi/release/yyos_boot.efi
make target/x86_64-unknown-none/release/yyos_kernel
make build
git diff --check
```

普通宿主目标直接检查 bootloader 二进制会触发该工程已有的 `no_std` `panic-unwind` 限制，因此 Bootloader 以真实的 `x86_64-unknown-uefi` 目标构建为准。

2. 启动与显示验证

QEMU 启动日志显示：

```
GOP framebuffer: 1280x800, stride 1280,  
format Bgr, address 0x80000000, size 0x3e8000  
Framebuffer Initialized: 1280x800, stride 1280, Bgr  
Enable IOApic: IRQ=1, CPU=0  
YYOS initialized.
```

系统随后正常加载 Shell，未发生页故障、死锁或重复中断。

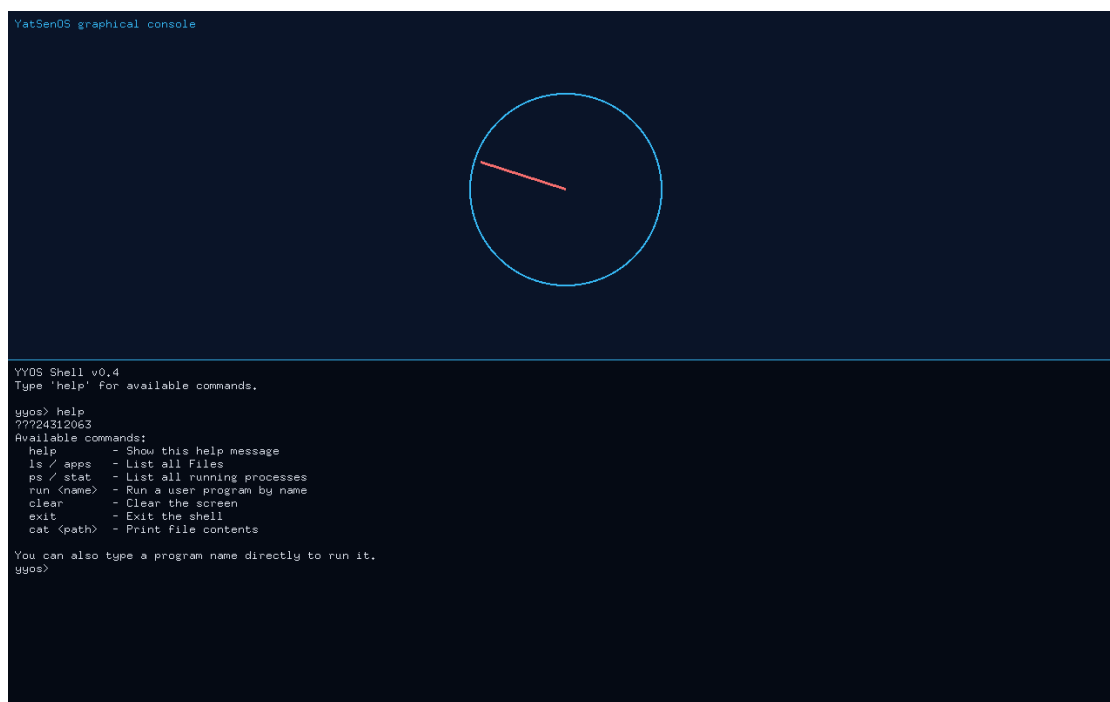


图 1 QEMU framebuffer 实际输出：上半屏动态图形区与下半屏 Shell

截图分辨率为 1280×800，检测到上下区域不同背景色以及文本、边框和时钟指针颜色，说明图形和字符确实由 framebuffer 驱动绘制，而不是来自宿主串口终端。

3. 键盘交互验证

测试通过 QEMU monitor 向 PS/2 键盘发送：

```
sendkey h-e-l-p-ret
```


串口和 VGA Shell 均显示输入回显，Shell 成功执行 `help` 并输出完整帮助菜单，随后返回 `yyos>` 提示符。这同时验证了：

1. IRQ1 正确路由到内核；
2. 扫描码状态机能够解析字符和回车；
3. 键盘输入进入共享 `INPUT_BUF`；
4. 用户态 `sys_read` 和 Shell 命令循环工作正常；
5. 输出能够同时到达 UART 与 framebuffer。

七. 问题与解决

1. framebuffer 初始化顺序

第一版显示驱动调用全局 `physical_to_virtual`，但 framebuffer 初始化早于物理地址模块，运行时会因为偏移尚未注册而 panic。最终改为将 `BootInfo` 中的 framebuffer 信息和物理偏移一起传入显示驱动，从接口上消除初始化顺序依赖。

2. stride 与可见宽度

framebuffer 每行的物理像素数不一定等于屏幕宽度。如果使用 width 计算地址，在某些模式下会出现斜行和越界。实现始终使用 `stride` 计算偏移，使用 width/height 进行可见区域裁剪。

3. 中断上下文锁竞争

时钟中断若阻塞等待 framebuffer 锁，而持锁代码又依赖中断或调度继续执行，会造成死锁。动画刷新使用 `try_lock`，锁被占用时丢弃一帧。图形动画允许丢帧，但输入输出不能因此停顿。

4. ANSI 控制序列

串口终端原先由宿主解释 ANSI 序列，framebuffer 只会看到普通字节。为兼容已有 Shell 和彩色 logger，驱动加入了小型状态机，仅实现工程实际使用的清屏、光标和颜色命令，避免引入完整终端模拟器的复杂度。

5. GUI 与串口同时使用

`-nographic` 会把显示设备重定向到终端，不适合作为 GUI 键盘验收入口。默认运行参数改成 `-serial stdio`，保留 QEMU 图形窗口，同时仍能在终端查看串口日志；另保留 `run-headless` 兼容自动化测试。

八. 思考与改进

1. 当前字体只覆盖 ASCII。后续可加入点阵中文字库或按需加载 PSF 字体。
2. 当前终端只实现实验所需 ANSI 子集。可以继续支持光标移动、前景/背景 256 色和局部擦除。
3. 当前动画由时钟中断直接绘制。更完整的系统应由显示服务或后台内核线程消费 tick 事件，在普通进程上下文重绘。
4. 当前绘制直接写显存，复杂界面可能闪烁。可以增加内存后备缓冲和 dirty rectangle，只刷新变化区域。
5. 当前键盘布局固定为 US 104 键。可将布局、Caps Lock 指示灯和扩展按键处理抽象为输入子系统。

九. 总结

本实验完成了从 UEFI GOP 到内核图形终端的完整链路。Bootloader 将 framebuffer 元数据传入内核，内核基于 `embedded-graphics` 实现像素、图元和字符绘制，并让日志及用户程序标准输出同时到达串口和 VGA。PS/2 键盘中断与串口共享输入队列，使 QEMU GUI 可以直接操作原有 Shell。

在必做内容之外，系统还实现了上下分屏和持续更新的图形时钟。最终真实 UEFI 目标构建、完整镜像构建、QEMU 启动、framebuffer 截图和键盘命令测试均通过。

十. 参考资料

1. YatSenOS 实验八任务：<https://ysos.gzti.me/labs/0x08/tasks/>。
2. UEFI Specification：Graphics Output Protocol。
3. `uefi 0.36.1` crate 文档。
4. `embedded-graphics 0.8.2` crate 文档。
5. `pc-keyboard 0.8.0` crate 文档。
6. OSDev Wiki：PS/2 Keyboard、VGA Hardware、Drawing in a Linear Framebuffer。